The Show Verse: plataforma web de gestión y descubrimiento de contenido audiovisual con integración de múltiples APIs

Trabajo de Fin de Grado

Convocatoria: Junio 2025

Alumno/a: Pablo Santos Conde

Tutor/a: Dr/Dra. D/Dña. Nombre y dos apellidos

Grado: Ingeniería del Software

Agradecimientos

Me gustaría agradecer a mi tutor de TFG por su orientación y disponibilidad a lo largo del proyecto. También a la comunidad de desarrolladores de las APIs de TMDb y Trakt, cuya documentación pública ha sido fundamental para el desarrollo de este trabajo.

Resumen

El presente Trabajo de Fin de Grado aborda el diseño y la implementación de *The Show Verse*, una aplicación web de gestión y descubrimiento de películas y series que permite a los usuarios unificar todo el contenido audiovisual en un solo lugar. La plataforma cuenta con secciones personalizadas para cada usuario, gestionadas a través de APIs externas.

La dispersión del contenido entre las distintas plataformas de *streaming* genera una experiencia de usuario fragmentada y dificulta el seguimiento del contenido consumido. *The Show Verse* soluciona este problema ofreciendo una plataforma que unifica el contenido de las principales páginas de puntuaciones y plataformas de *streaming* haciendo uso de APIs.

El trabajo consiste en el desarrollo completo de una aplicación web en Next.js que integra las APIs de The Movie Database (TMDb), Trakt.tv y OMDb, además de JustWatch para la disponibilidad en plataformas de *streaming* y Plex para el acceso a contenido en un servidor local. Se ha implementado un flujo de autenticación OAuth 2.0 completo con Trakt.tv y TMDb para permitir la sincronización del historial, los favoritos, la lista de pendientes, el calendario de estrenos y las valoraciones del usuario. Además, se ha diseñado un sistema de caché en múltiples niveles que reduce en un 70–80 % el número de peticiones a APIs externas en las vistas de listas.

Índice general

Índice de figuras

Índice de tablas

Abstract

This Final Degree Project presents the design and implementation of *The Show Verse*, a web application for the management and discovery of films and TV series that allows users to unify all audiovisual content in a single platform. The application integrates the TMDb, Trakt.tv and OMDb APIs for metadata and personal tracking, JustWatch for streaming availability, and Plex for access to content on a local media server. A complete OAuth 2.0 authentication flow with Trakt.tv has been implemented, together with a multi-level cache strategy that reduces external API calls by up to 80 % in list views. The result is a responsive web platform with fluid animations, three display modes (Grid, List and Compact), and Lighthouse scores of 92/100 in performance and 100/100 in SEO.

Glosario

API *Application Programming Interface*. Interfaz de programación de aplicaciones.

CDN *Content Delivery Network*. Red de distribución de contenidos.

CLS *Cumulative Layout Shift*. Métrica de estabilidad visual de Core Web Vitals.

CSRF *Cross-Site Request Forgery*. Tipo de ataque web que fuerza acciones no deseadas.

FID *First Input Delay*. Tiempo de respuesta al primer input del usuario.

GraphQL Lenguaje de consulta para APIs desarrollado por Meta.

httpOnly Atributo de cookie que impide su acceso desde JavaScript del navegador.

ISR *Incremental Static Regeneration*. Estrategia de caché de Next.js.

JWT *JSON Web Token*. Estándar para la transmisión segura de información entre partes.

LCP *Largest Contentful Paint*. Métrica de velocidad de carga de Core Web Vitals.

OAuth *Open Authorization*. Protocolo estándar de delegación de acceso.

OMDb *Open Movie Database*. Base de datos de películas con datos de IMDb.

REST *Representational State Transfer*. Estilo arquitectónico para servicios web.

RSC *React Server Components*. Componentes React que se ejecutan exclusivamente en servidor.

SPA *Single Page Application*. Aplicación web de una sola página.

SPR *Serverless Page Rendering*. Renderizado de páginas sin servidor propio.

SSR *Server-Side Rendering*. Renderizado de páginas en el servidor.

TMDb *The Movie Database*. Base de datos de películas y series con API pública.

TTL *Time To Live*. Tiempo de vida de un elemento en caché.

XSS *Cross-Site Scripting*. Tipo de ataque de inyección de código en el navegador.

Capítulo 1
Introducción

1.1 Justificación y contexto

En los últimos años, el consumo de películas y series ha cambiado radicalmente. Hoy en día la mayoría del contenido se distribuye a través de plataformas de *streaming* como Netflix, HBO Max, Disney+, Amazon Prime Video o Movistar+, cada una con su propio catálogo independiente, lo que genera un problema habitual para cualquier usuario: el contenido está disperso en múltiples servicios y no existe ninguna herramienta que permita gestionarlo todo desde un único lugar.

Esto provoca situaciones como perder el seguimiento de una serie que se dejó a medias, no recordar qué episodios se han visto, tener que buscar en varios sitios a la vez si una película está disponible en *streaming*, o simplemente no tener un registro de lo que ya se ha visto. Las plataformas de *tracking* que existen actualmente, como Trakt.tv o Letterboxd, resuelven parte del problema pero presentan sus propias limitaciones con falta de funcionalidades.

The Show Verse nace como respuesta a esa necesidad: una plataforma web que centraliza toda la gestión del contenido audiovisual, integrando los datos de múltiples fuentes mediante APIs externas y ofreciendo al usuario una experiencia completa y funcional.

Desde el punto de vista técnico, el proyecto aborda varios retos propios del desarrollo web actual: la integración de APIs REST de terceros con distintos esquemas de autenticación y formatos de datos, como la implementación del protocolo OAuth 2.0 para el acceso con Trakt.tv, el diseño de una estrategia de caché en múltiples niveles para optimizar el rendimiento, y la construcción de una interfaz responsiva con animaciones fluidas.

1.1.1 Motivación

El punto de partida de este trabajo es la experiencia personal como consumidor habitual de películas y series en múltiples plataformas de *streaming*. La dispersión del catálogo disponible entre los diferentes servicios provocaba con frecuencia situaciones de dificultad para recordar el historial

de películas y series y la puntuación que se le había dado a cada una, además de tener que buscar las puntuaciones y críticas en distintas webs independientes.

Las aplicaciones existentes en el mercado (Letterboxd, Trakt.tv, Simkl) ofrecen soluciones parciales, pero ninguna combina todos los datos y funcionalidades realmente necesarios a la hora de gestionar este tipo de contenido. Trakt.tv, la más completa en cuanto a API y funcionalidades de *tracking*, presenta una interfaz datada que resulta poco motivadora para el uso diario. Letterboxd, más elegante visualmente, carece de soporte completo para series de televisión y para otras funcionalidades importantes.

1.2 Planteamiento del problema

El problema abordado en este trabajo puede descomponerse en varias categorías técnicas.

1. Integración de fuentes de datos. Los datos necesarios para una plataforma de gestión de contenido audiovisual completa no están disponibles en una única API. TMDb proporciona los metadatos de películas y series (sinopsis, reparto, imágenes, trailers), además de las puntuaciones y listas de usuario. Trakt.tv gestiona el *tracking* personal del usuario (historial de visionado, sección en progreso y estadísticas). OMDb complementa con las puntuaciones de IMDb y Rotten Tomatoes. Cada API tiene su propio esquema de autenticación, formato de respuesta, límites de uso e identificadores.

2. Autenticación y gestión de sesiones. La funcionalidad de sincronización con Trakt requiere que el usuario delegue permisos en la aplicación mediante OAuth 2.0, lo que implica implementar el flujo completo de autorización (redirección, intercambio de código por token, almacenamiento seguro, refresco automático), sin exponer en ningún momento las credenciales del usuario ni las claves privadas de la aplicación.

3. Rendimiento y experiencia de usuario. Este tipo de aplicación maneja grandes volúmenes de imágenes de alta resolución y realiza múltiples llamadas a APIs externas con límites de peticiones en algunos casos reducidos. Es importante que haya un equilibrio entre datos almacenados en caché y aquellos que se muestran en tiempo real.

4. Sincronización de estado entre servidor y cliente. Las acciones del usuario (marcar como visto, añadir a favoritos, puntuar) deben reflejarse de forma inmediata en la interfaz sin esperar la confirmación del servidor, pero manteniendo la coherencia en caso de error.

1.3 Objetivos del trabajo

Objetivo general: diseñar e implementar una aplicación web completa y funcional para la gestión personalizada y el descubrimiento de nuevas películas y series, que integre múltiples APIs externas y ofrezca una experiencia de usuario centralizada.

Objetivos específicos:

1) Implementar la integración con la API de TMDb para obtener los metadatos completos de películas y series (sinopsis, características, imágenes, vídeos, temporadas y episodios).

2) Desarrollar un sistema de autenticación OAuth 2.0 completo con Trakt.tv, incluyendo almacenamiento seguro de tokens en cookies, actualización automática y revocación.

3) Construir un módulo de gestión personal (favoritos, pendientes, historial de visionado y en progreso y puntuaciones de usuario) con sincronización bidireccional con Trakt y TMDb.

4) Implementar un sistema de seguimiento por episodios y temporadas para las series.

5) Diseñar una interfaz de usuario responsiva y accesible que ofrezca múltiples modos de visualización (Grid, List, Compact) con transiciones fluidas y compatibilidad con dispositivos móviles.

6) Optimizar el rendimiento de la aplicación, incluyendo un sistema de caché en cliente para las puntuaciones de IMDb, Trakt y OMDb en las listas de favoritos y pendientes.

7) Desplegar la aplicación en un entorno de producción como Vercel.

Capítulo 2
Estado de la cuestión

El presente estado de la cuestión aborda el contexto tecnológico y de mercado en el que se enmarca el proyecto. Se comienza con una visión del sector de las plataformas de *streaming* y sus implicaciones para el usuario; a continuación se analizan las tecnologías de desarrollo web actuales que hacen posible la solución propuesta; y se finaliza examinando las plataformas existentes más relevantes.

2.1 Marco teórico del trabajo

2.1.1 Las plataformas de streaming y sus consecuencias para el usuario

La transición del modelo tradicional al *streaming* bajo demanda se inició con el lanzamiento del servicio de Netflix en 2007, aunque su consolidación definitiva se produjo a partir de 2019, con la irrupción simultánea de Disney+, Apple TV+ y HBO Max. En 2024, el usuario medio en mercados como España o Estados Unidos tiene acceso simultáneo a entre 3 y 5 servicios de *streaming* distintos, según datos de CNMC y Statista (CNMC, 2024; Statista, 2024).

Esta fragmentación genera una sobrecarga de elección en cada usuario, lo que provoca que inviertan una gran cantidad de tiempo en decidir qué ver, olviden mantener un registro de lo visto y pierdan el seguimiento de las series que consumen de forma no continua (Schwartz, 2004).

La necesidad de una capa de abstracción sobre las plataformas de *streaming* ha generado el desarrollo de aplicaciones conocidas como **media trackers**.

2.1.2 Protocolos de autenticación: OAuth 2.0

OAuth 2.0 es el estándar que permite a una aplicación actuar en nombre de un usuario en un servicio externo sin que ese servicio tenga que proporcionarle su contraseña (Hardt, 2012). En *The Show Verse* se utiliza para conectar la cuenta de Trakt.tv del usuario con la aplicación.

El flujo implementado (Authorization Code) funciona de la siguiente manera:

1) Cuando el usuario pulsa «Conectar con Trakt», la ruta `/api/trakt/auth/start` genera un código aleatorio de seguridad (`state`) para evitar ataques CSRF, construye la URL de autorización de Trakt y redirige al usuario.

2) El usuario introduce sus credenciales en Trakt y concede permisos. Trakt redirige de vuelta a `/api/trakt/auth/callback` incluyendo un `code` de un solo uso y el mismo `state`.

3) La ruta de *callback* verifica el `state` e intercambia el `code` por un `access_token` y un `refresh_token` directamente desde el servidor.

4) Ambos tokens se guardan en cookies `httpOnly` con los atributos `Secure` y `SameSite=Lax`, de forma que el JavaScript del navegador no puede leerlos en ningún momento.

5) Cuando el `access_token` expira, la aplicación lo renueva automáticamente usando el `refresh_token`.

2.1.3 APIs REST

Las APIs utilizadas en este proyecto siguen el estilo REST (Fielding, 2000) con las siguientes especificidades:

TMDb API v3 es una API REST pública con autenticación por API Key. Proporciona acceso a su base de datos de más de 800.000 películas y 150.000 series de televisión, con soporte para más de 40 idiomas. Su límite de peticiones es de 40 peticiones por 10 segundos para cuentas gratuitas (The Movie Database, 2024).

Trakt.tv API v2 combina autenticación por API Key para *endpoints* públicos con OAuth 2.0 Bearer Token para *endpoints* autenticados. Un aspecto técnico relevante es el sistema de identificadores: Trakt mantiene sus propios slugs e IDs, pero acepta identificadores de IMDb y TMDb mediante el *endpoint* de búsqueda por ID externo (Trakt, 2024).

2.1.4 Rendimiento web

Las estrategias de optimización más relevantes para este proyecto incluyen: Server-Side Rendering para reducir el LCP, el componente `<Image>` de Next.js para optimización automática de imágenes (conversión a WebP, *lazy loading*, reserva de espacio para prevenir CLS), y el *code splitting* automático por ruta para reducir el tiempo de carga inicial (Google Developers, 2024a, 2024b).

2.1.5 Diseño de interfaz

El diseño de *The Show Verse* se basa en el uso de los *backdrops* y *posters* obtenidos de TMDb como elemento principal de la interfaz.

Modo oscuro y backdrops de fondo. El color de base de la aplicación es el negro (`#000000`). Las imágenes de fondo se despliegan a pantalla completa con un gradiente en los bordes. Los elementos superpuestos utilizan un fondo semi-transparente con desenfoque del contenido detrás (`backdrop-filter: blur`), siguiendo el patrón de *glassmorphism* (Babich, 2021).

Animaciones de entrada y transiciones. Las animaciones se gestionan mediante Framer Motion (Framer, 2024), usando `AnimatePresence` para la entrada y salida de componentes. Para listas largas (más de 30 elementos), el retraso entre tarjetas se reduce automáticamente para que la animación no resulte lenta.

2.2 Trabajos relacionados

Letterboxd (Letterboxd, 2024) (fundado en 2011, Auckland, Nueva Zelanda) es la plataforma de referencia para el registro y la crítica cinematográfica personal. Con más de 20 millones de usuarios registrados en 2024, su principal fortaleza es la componente social y el sistema de reseñas. Su limitación más significativa para los propósitos de este trabajo es la ausencia de soporte para el seguimiento de series de televisión por episodio.

Trakt.tv (fundado en 2010) es la plataforma más completa en cuanto a funcionalidades de *tracking*, con soporte completo para películas y series a nivel de episodio. Su característica diferencial es la API pública v2, documentada y con límites de uso generosos para desarrolladores registrados. Sin embargo, su interfaz de usuario presenta un diseño anticuado que no ha evolucionado significativamente en la última década.

Simkl (fundado en 2012) ofrece funcionalidades similares a Trakt con una interfaz algo más moderna, pero con un ecosistema de API menos estable y una comunidad de usuarios significativamente menor.

JustWatch (fundado en 2014) se especializa en la localización de contenido entre plataformas de *streaming*, pero carece de funcionalidades de *tracking* personal.

La comparación entre estas plataformas pone de manifiesto una oportunidad en el mercado: ninguna combina la funcionalidad de Trakt (seguimiento por episodio, sincronización, valoraciones) con una interfaz de usuario moderna y responsiva comparable a las propias plataformas de *streaming*. *The Show Verse* se posiciona precisamente en este punto.

Capítulo 3
Aspectos metodológicos

El desarrollo de *The Show Verse* se ha abordado mediante una **metodología iterativa basada en el ciclo de vida ágil**, adaptada a un proyecto individual. La elección de este enfoque frente a las metodologías en cascada se justifica por la naturaleza exploratoria del proyecto: la integración con APIs de terceros implica incertidumbres técnicas que no siempre se pueden anticipar en una fase de análisis, y la prioridad entre funcionalidades puede variar a medida que se comprende mejor el problema.

El proceso de desarrollo se articula en iteraciones de aproximadamente una semana de duración. Al inicio de cada iteración se seleccionan del *backlog* las funcionalidades a abordar, priorizadas según tres criterios:

- **Dependencia técnica:** las funcionalidades que sirven de base a otras (autenticación, cliente de API) se abordan primero.
- **Valor aportado al usuario:** se priorizan las funcionalidades más visibles y relevantes para la experiencia de uso.
- **Riesgo técnico:** las integraciones más inciertas se adelantan para conocer cuanto antes sus restricciones.

3.1 Metodología

3.1.1 Criterios de integración de APIs

La selección de las APIs externas ha seguido los siguientes criterios:

- **Datos completos:** TMDb fue seleccionada frente a otras alternativas (The Open Movie Database, Wikidata) por ofrecer la mayor cobertura de contenido, soporte en diferentes idiomas, galería de imágenes y datos de *streaming*, además de disponer de la documentación más completa y una comunidad de desarrolladores activa.

- **Capacidad de tracking:** Trakt.tv fue seleccionada como plataforma de sincronización por su API pública v2 bien documentada, su soporte completo de series a nivel de episodio y por ser la plataforma de *tracking* más referenciada por la comunidad de desarrolladores independientes.
- **Complementariedad:** OMDb fue seleccionada únicamente para los datos de *rating* de IMDb y Rotten Tomatoes, que TMDb no proporciona directamente.

3.1.2 Criterios de decisión para la autenticación

El problema de la autenticación OAuth 2.0 admitía varias opciones de implementación en Next.js. Se descartó el uso de librerías de autenticación genéricas como NextAuth.js / Auth.js por las siguientes razones:

- Trakt.tv no es un proveedor OAuth soportado de forma nativa por estas librerías.
- La personalización necesaria (refresco de tokens específico de Trakt, intercambio de códigos con *headers* personalizados) habría requerido más configuración que implementar el flujo directamente.
- La implementación propia ofrece mayor control sobre el manejo de errores y el almacenamiento de tokens.

Se optó por implementar el flujo Authorization Code de forma manual mediante **API Routes de Next.js**, almacenando los tokens en cookies `httpOnly` con atributos `Secure` y `SameSite=Lax`. Esta decisión garantiza que los tokens no sean accesibles desde JavaScript del navegador (protección frente a XSS) y que no se envíen en peticiones *cross-site* no intencionadas (protección parcial frente a CSRF).

3.1.3 Estrategia de caché

Para cada tipo de dato gestionado en la aplicación se ha definido una estrategia de caché en función de la frecuencia de actualización del dato en origen:

Esta diferenciación permite reducir el número de llamadas a APIs externas en un estimado del 70–80 % para los datos de catálogo, mientras se garantiza la actualización de los datos de usuario.

3.2 Tecnologías empleadas

Next.js 16 (Framework principal). Seleccionado frente a otras alternativas (Remix, SvelteKit, Nuxt 3) por ser el framework React con mayor adopción industrial (>40 % de cuota entre

Tipo de dato	**Almacenamiento**	**TTL**
Catálogo general (top rated, populares)	ISR servidor	10 minutos
Detalles de película/serie	ISR servidor	1 hora
Imágenes TMDb	CDN Vercel	1 hora
Datos OMDb (proxy servidor)	Map en proceso	24 h / 60 s
Datos OMDb en cliente	sessionStorage	24 horas
Puntuaciones IMDb y Trakt en listas	localStorage	30 días
Rating personal del usuario	Map en módulo	10 min / 45 s
Estado personal del usuario	No-cache	En cada petición

Tabla 3.1: Estrategia de caché por tipo de dato

frameworks React según npm trends 2024), la mejor integración con Vercel y la documentación más extensa. El App Router de Next.js 13+ es la única opción que implementa React Server Components de forma estable y con soporte oficial (Vercel, 2024).

React 19. La elección de React como biblioteca de UI es consecuencia directa de la elección de Next.js. React 19 introduce mejoras en el manejo de acciones de servidor y la promesa nativa en componentes de cliente (Meta Open Source, 2024).

Tailwind CSS 4. Seleccionado frente a CSS Modules, Styled Components o Emotion por el paradigma *utility-first* que acelera el desarrollo sin sacrificar la personalización. La versión 4, basada en PostCSS con motor Rust nativo, elimina el PurgeCSS separado e incorpora mejoras sustanciales de rendimiento en el tiempo de compilación.

Framer Motion 12. La elección de Framer Motion frente a otras librerías de animación (React Spring, CSS Animations puras) se basa en su modelo declarativo basado en variantes, su soporte nativo para animaciones de *layout* y su integración con el ciclo de vida de los componentes React mediante `AnimatePresence` (Framer, 2024).

TypeScript 5. Configurado en modo de verificación estricta para la configuración del proyecto y los tipos de datos de las respuestas de API.

Capítulo 4
Desarrollo del trabajo

En este capítulo se describe el desarrollo completo de *The Show Verse*, desde la arquitectura general del sistema hasta la implementación de cada módulo funcional y la integración con las APIs externas. Es la parte más importante del TFG y recoge las decisiones técnicas tomadas durante el proceso de construcción de la plataforma.

4.1 Arquitectura del sistema

4.1.1 Estructura de capas

El sistema se estructura en tres capas diferenciadas, implementadas en una única base de código Next.js:

Capa de presentación (Client Components). Componentes React que se ejecutan y se hidratan en el navegador. Gestionan el estado local de la interfaz, las interacciones del usuario y las peticiones a la capa intermedia. Los componentes de mayor complejidad son `MainDashboardClient.jsx` (gestión del dashboard completo), `DetailsClient.jsx` (página de detalle de película y serie) y `DiscoverClient.jsx` (módulo de búsqueda).

Capa de servidor (Server Components + API Routes). Los Server Components de Next.js realizan la búsqueda inicial de datos directamente en el servidor, sin coste de JavaScript para el cliente. Las API Routes actúan como *proxy* hacia las APIs externas, añadiendo los tokens de autenticación en el servidor y transformando los datos al formato que consume el *frontend*.

Capa de datos (APIs externas). TMDb, Trakt.tv, OMDb, JustWatch y Plex como fuentes para los datos de catálogo y del usuario respectivamente.

La separación entre estas capas se hace explícita en la convención de nombres del proyecto: los archivos terminados en `Client.jsx` son siempre Client Components (marcados con `üse client"`), mientras que los archivos `page.jsx` y los de la carpeta `api/` son siempre Server Components o API Routes.

4.1.2 Estructura de carpetas

El proyecto sigue la convención del App Router de Next.js 13+. La estructura principal se organiza en cuatro directorios bajo `src/`:

- `src/app/` — Rutas, páginas y API Routes. Cada subdirectorio corresponde a una ruta de la aplicación.
- `src/components/` — Componentes React reutilizables. Los de mayor peso son `DetailsClient.jsx` (∼331 KB) y `MainDashboardClient.jsx` (∼102 KB).
- `src/lib/` — Lógica de negocio, clientes de API y utilidades. Incluye los módulos `tmdb.js`, `traktClient.js`, `justwatch.js` y `plex/auth.js`.
- `src/context/` — Contextos globales de React para el estado compartido entre componentes.

Las rutas principales de la aplicación son:

Ruta	**Descripción**
`/`	Dashboard principal con carrusel y secciones
`/details/[type]/[id]`	Página de detalle de película o serie
`/favorites`	Lista de favoritos del usuario (TMDb)
`/watchlist`	Lista de pendientes del usuario (TMDb)
`/history`	Historial de visionado (Trakt)
`/in-progress`	Series en progreso (Trakt)
`/stats`	Estadísticas del usuario (Trakt)
`/calendar`	Calendario de estrenos y próximos episodios
`/discover`	Búsqueda y filtrado avanzado de contenido
`/biblioteca`	Librería personal unificada
`/lists`	Listas de TMDb, colecciones y listas de Trakt
`/movies`	Exploración de catálogo de películas
`/series`	Exploración de catálogo de series
`/details/person/[id]`	Ficha de actor o directora

Tabla 4.1: Rutas principales de la aplicación

4.2 Implementación del sistema de autenticación OAuth 2.0

La implementación del flujo OAuth 2.0 con Trakt.tv es el núcleo técnico más delicado del proyecto. El flujo se articula en cuatro API Routes de Next.js:

Inicio del flujo (`/api/trakt/oauth`): construye la URL de autorización de Trakt con los parámetros `client_id`, `redirect_uri` y `response_type=code`, y redirige al usuario. Este *endpoint* se invoca al pulsar el botón «Conectar con Trakt» de la Navbar.

Callback (`/api/trakt/auth/callback`): recibe el `authorization_code` de Trakt, lo intercambia por `access_token` y `refresh_token` mediante una petición POST servidor-a-servidor a `https://api.trakt.tv/oauth/token`, y almacena ambos tokens en cookies `httpOnly`. A continuación redirige al usuario a la página de inicio.

Estado (`/api/trakt/auth/status`): lee las cookies de sesión y, si existe `access_token` válido, hace una petición a `/users/me` de Trakt para obtener el perfil del usuario. Si el token está expirado, intenta refrescarlo usando el `refresh_token` antes de declarar al usuario como no autenticado.

Desconexión (`/api/trakt/auth/disconnect`): revoca el token en Trakt mediante la API `/oauth/revoke` y elimina las cookies de sesión.

Este diseño garantiza que ni el `client_secret` de Trakt ni los `access_token` del usuario sean accesibles desde el JavaScript del navegador en ningún momento del flujo.

4.3 Integración con las APIs externas

4.3.1 The Movie Database (TMDb) API v3

El cliente de TMDb (`lib/api/tmdb.js`) encapsula toda la lógica de acceso a la API v3. La función central `tmdb(path, params, options)` implementa:

- Un **constructor de URL unificado** que añade automáticamente la API Key y el idioma por defecto (`es-ES`).
- Un **mecanismo de timeout** mediante `AbortController` (8 segundos por defecto) para evitar que peticiones lentas bloqueen la interfaz.
- **Comportamiento de caché diferenciado**: `force-cache` con revalidación ISR en servidor, `no-store` en cliente.

- **Gestión granular de errores**: los errores 404 retornan `null` silenciosamente; otros errores se loguean y también retornan `null`.

Sobre esta función base se construyen más de 40 funciones nombradas por dominio (`fetchTopRatedMovies`, `getDetails`, `getCredits`, `getWatchProviders`, `getActorDetails`, etc.) que constituyen la API pública del módulo.

Un aspecto técnico importante es la resolución de imágenes: la función `getLogos` obtiene el logo oficial de un título aplicando un criterio de selección que prioriza logos en español, luego en inglés, luego sin idioma, seleccionando el de mayor número de votos dentro de cada grupo.

La Tabla **??** recoge los *endpoints* principales utilizados.

Función	**Endpoint TMDb**
`fetchTopRatedMovies()`	`GET /movie/top_rated`
`fetchTrendingMovies()`	`GET /trending/movie/week`
`getDetails(type, id)`	`GET /{type}/{id}?append_to_response=external`
`getLogos(type, id)`	`GET /{type}/{id}/images`
`getCredits(type, id)`	`GET /{type}/{id}/credits`
`getWatchProviders(type, id)`	`GET /{type}/{id}/watch/providers`
`getRecommendations(type, id)`	`GET /{type}/{id}/recommendations`
`markAsFavorite({...})`	`POST /account/{id}/favorite`
`markInWatchlist({...})`	`POST /account/{id}/watchlist`
`fetchFavoritesForUser(...)`	`GET /account/{id}/favorite/movies|tv`
`fetchWatchlistForUser(...)`	`GET /account/{id}/watchlist/movies|tv`

Tabla 4.2: Principales funciones del cliente TMDb y sus *endpoints*

4.3.2 OMDb API (Open Movie Database)

OMDb es la única fuente pública que permite obtener el *rating* de IMDb, el número de votos y las puntuaciones de Rotten Tomatoes y Metacritic por `imdbId`. Para proteger la API Key, todas las peticiones a OMDb pasan por la API Route interna `/api/omdb`.

El *proxy* implementa un **sistema de deduplicación** (`inflight Map` global): si múltiples tarjetas solicitan simultáneamente el mismo `imdbId`, solo se realiza una petición real a OMDb. Las respuestas exitosas se cachean 24 horas en el proceso de Node.js; las fallidas, 60 segundos

para reintentar pronto. La cabecera `Cache-Control: public, s-maxage=86400` permite adicionalmente que la CDN de Vercel cachee las respuestas.

4.3.3 Proxy de Trakt y transformación de datos

Las API Routes de Trakt actúan como intermediario que resuelve dos problemas estructurales:

Problema de identificadores. El *frontend* trabaja exclusivamente con `tmdbId`, mientras que muchos *endpoints* de Trakt requieren sus propios identificadores (`traktId`, `slug`). Las API Routes realizan internamente la traducción mediante el *endpoint* de búsqueda por ID externo de Trakt (`/search/tmdb/{id}?type=movie|show`).

Problema de enriquecimiento de datos. Las respuestas de Trakt incluyen datos de *tracking* pero no imágenes. Las API Routes que devuelven listas de contenido (historial, favoritos, pendientes) realizan peticiones paralelas a TMDb para obtener las URLs de *poster* y *backdrop*, enriqueciendo la respuesta antes de enviarla al cliente.

La tabla **??** recoge los *endpoints* de Trakt utilizados por módulo funcional.

Endpoint Trakt	**Descripción**
`GET /sync/history`	Historial completo de películas y episodios vistos
`POST /sync/history`	Registrar película/episodio como visto
`DELETE /sync/history`	Desmarcar como visto
`GET /sync/watched/shows`	Estado de visionado de todas las series
`GET /sync/watchlist`	Lista de pendientes del usuario
`POST /sync/ratings`	Añadir valoración (1–10)
`GET /users/me/stats`	Estadísticas globales del usuario
`GET /calendars/my/shows`	Próximos episodios de series seguidas
`GET /search/tmdb/{id}`	Búsqueda de item por ID de TMDb

Tabla 4.3: Principales *endpoints* de Trakt utilizados

4.3.4 JustWatch (API GraphQL no oficial)

JustWatch no tiene una API oficial pública. El cliente (`lib/api/justwatch.js`) utiliza los mismos *endpoints* GraphQL que la web de JustWatch, enviando peticiones `POST` a `https://apis.justwatch.com/graphql`.

Se utilizan dos *queries* GraphQL:

1) **Búsqueda del título** (`GetSearchTitles`): recibe el título y tipo de contenido y devuelve el `nodeId` interno de JustWatch.

2) **Detalles con ofertas** (`GetTitleDetails`): recibe el `nodeId` y devuelve las ofertas de *streaming* disponibles para España, incluyendo el tipo de monetización (`FLATRATE`, `RENT`, `BUY`), la URL directa al contenido en la plataforma y los datos del proveedor.

La función principal `getStreamingProviders(title, type, year)` filtra las ofertas de tipo `FLATRATE` (suscripción), les asigna el logo de la plataforma mediante un mapeo estático a los logos de TMDb y devuelve la URL directa al contenido.

4.3.5 Plex Media Server

La integración con Plex permite a los usuarios que tienen una biblioteca local abrir directamente cualquier película o serie desde *The Show Verse* en su servidor Plex, sin tener que buscar el contenido manualmente.

Autenticación. El módulo `lib/plex/auth.js` implementa dos modos:

- **Modo simple:** se configura `PLEX_TOKEN` en variables de entorno y se usa directamente en cada petición.
- **Modo JWT:** se configura una clave privada RSA o Ed25519 (`PLEX_JWT_PRIVATE_KEY`). La aplicación firma JWTs con la clave privada, los intercambia por tokens de acceso en la API de Plex y los renueva automáticamente antes de su expiración. La deduplicación de refrescos concurrentes se gestiona mediante un `Promise` compartido (`state.refreshPromise`).

API Route interna (`GET /api/plex`). Recibe `title`, `type`, `year`, `imdbId` y `tmdbId` como parámetros y devuelve varios formatos de URL para el acceso al contenido desde diferentes plataformas:

- `plexWebUrl`: URL para la interfaz web de Plex en escritorio.
- `plexUniversalUrl`: `https://watch.plex.tv/{type}/{slug}`, un enlace universal que abre la app nativa en iOS y Android.
- `plexSlugUrl`: `plex://{type}/{slug}`, *deep link* nativo de la aplicación.
- `plexAndroidSlugIntentUrl`: formato `intent://` para Chrome en Android.

El slug para las URLs universales se resuelve consultando `https://metadata.provider.plex.tv/li`

4.4 Implementación de los módulos funcionales

Esta sección documenta en profundidad los 15 módulos funcionales de la aplicación, explicando su arquitectura, flujos de datos, patrones de optimización y características especiales.

4.4.1 1. Dashboard principal (/)

Propósito: punto de entrada de la aplicación, mostrando un compendio curado de contenido audiovisual organizado en 12 secciones temáticas.

Arquitectura Server Component: la página de inicio (`src/app/page.jsx`) es un Server Component que ejecuta en paralelo múltiples peticiones a TMDb mediante `Promise.all()`, reduciendo el tiempo total de carga a la latencia de la petición más lenta (típicamente 400–600 ms) en lugar de la suma de todas (4–8 s).

Las secciones del dashboard (obtidas del endpoint `/trending/movie/week`, `/movie/top_rated`, `/discover/movie?with_genres=28`, etc.) se detallan en la Tabla **??**:

Sección	Endpoint TMDb	Descripción
Hero	`/trending/movie/week`	Carrusel de películas de este mes
Top Películas	`/movie/top_rated`	Películas mejor valoradas de todos los tiempos
Tendencias	`/trending/movie/week`	Películas populares esta semana
Populares	`/movie/popular`	Contenido con mayor momentum
Acción	`/discover/movie?with_genres=28`	Películas de acción clásicas
Dramas	`/discover/movie?with_genres=18`	Películas dramáticas imprescindibles
Series Populares	`/tv/popular`	Series trending del momento
Series Top Rated	`/tv/top_rated`	Series clásicas mejor valoradas
Series Trending	`/trending/tv/week`	Series en tendencia esta semana
Emitiéndose Hoy	`/tv/airing_today`	Episodios con emisión real-time
En Emisión	`/tv/on_the_air`	Series con temporada activa
Favoritos (auth)	`/account/{id}/favorite/movies`	Favoritos personales del usuario

Tabla 4.4: Secciones del dashboard y sus fuentes de datos

Selección de logotipos: cada película del carrusel hero requiere un logo oficial. La función `getLogos()` realiza `GET /{type}/{id}/images?language=es,en,null`, aplicando estrategia de prioridad: idioma español → inglés → sin idioma, seleccionando dentro de cada grupo el logo con mayor número de votos.

Componente cliente (`MainDashboardClient.jsx`, 102 KB): gestiona:

- **Carrusel del hero:** autoplay con intervalo de 6 s, pausa al *hover*, botones de navegación, indicadores visuales. Implementado con Swiper.js.
- **Sticky scroll de la Navbar:** al superar umbral de 100 px de desplazamiento, la barra de navegación transiciona de transparente a glassmorphism (`bg-black/70 backdrop-blur-12px`).
- **Animaciones de entrada escalonada:** cada sección entra progresivamente mediante `staggerChildren: 0.1` de Framer Motion, creando efecto cascada.
- **Secciones condicionales:** los favoritos y series en progreso solo se muestran si el usuario está autenticado.

Rendimiento: ISR con `revalidate = 1800` (30 minutos). Lazy loading de imágenes mediante componentes `<Image>` de Next.js con placeholder blur.

4.4.2 2. Página de detalle (`/details/[type]/[id]`)

Tamaño: `DetailsClient.jsx` tiene 331 KB, siendo el componente más grande del proyecto. Complejidad muy alta.

Arquitectura Server Component: obtiene en paralelo:

- Metadatos del título (con `append_to_response=external_ids` para obtener IDs externos en una sola petición)
- Logos oficiales mediante `getLogos()`
- Créditos (reparto y equipo técnico)
- Recomendaciones de títulos similares
- Proveedores de *streaming* por país
- Vídeos (trailers, teasers)
- Reseñas de la comunidad
- Estado de la cuenta del usuario (favorito, watchlist, valoración) si autenticado

Sistema de pestañas: el componente cliente gestiona cinco pestañas:

1) **Información general:** sinopsis, reparto principal, datos técnicos (duración, idioma, presupuesto para películas, recaudación).

2) **Temporadas/Episodios** (solo series): lista expandible de temporadas, cada una mostrando sus episodios con estado de visionado.

3) **Multimedia:** trailers, teasers y clips embebidos de YouTube.

4) **Recomendaciones:** títulos similares sugeridos por TMDb en carrusel horizontal.

5) **Reseñas:** críticas de la comunidad TMDb con puntuación y fecha.

Barra de puntuaciones (ScoreboardBar.jsx): centraliza rating de múltiples fuentes en una sola barra visual:

- **TMDb:** obtenido del Server Component, sin petición adicional en cliente. Muestra % (ej. 78 %).
- **IMDb:** obtenido de OMDb mediante `/api/omdb`, cacheado en `sessionStorage`. Solo se carga si usuario expande sección. Muestra rating numérico (ej. 8.2/10).
- **Trakt:** obtenido de `/api/trakt/scoreboard`, también cacheado. Muestra puntuación comunitaria (%) y permite al usuario autenticado votar 1–10 estrellas con actualización optimista.
- **Rotten Tomatoes / Metacritic:** opcionales, si disponibles de OMDb.

Gestión optimista de estado: al hacer clic en «Favorito», «Watchlist» o estrellas de valoración, el estado visual se actualiza INSTANTÁNEAMENTE. En background se envía la petición a TMDb/Trakt. Si falla, el estado se revierte automáticamente.

Integración Plex: si el usuario ha configurado servidor Plex, se muestra botón «Ver en Plex» con link directo al contenido en su biblioteca local.

Disponibilidad en streaming: se muestran simultáneamente proveedores de TMDb (cacheados 1 hora) y JustWatch (con link directo y búsqueda por plataforma).

4.4.3 3. Seguimiento episódico (series)

Complejidad: muy alta. Uno de los sistemas más complejos del proyecto.

Representación de estado: el visionado por episodio se indexa en `watchedBySeason`:

```
{
  "1": { "1": true, "2": true, "3": false },
  "2": { "1": false, "2": true },
  ...
}
```

Obtenido de Trakt mediante `GET /sync/watched/shows/{id}`, transformado internamente por la API Route `/api/trakt/show/watched`.

Componente SeasonDetailsClient.jsx: para cada temporada, renderiza:

- Grid o lista de episodios mostrando: número, nombre, fecha de aire, puntuación TMDb, estado de visionado.
- Toggle individual de visto/no visto con actualización optimista.
- Botón «Marcar temporada completa»: en lugar de N peticiones (una por episodio), construye lista de todos no vistos y envía 1 petición lote a Trakt.
- Porcentaje de progreso visual como barra de progreso.

Componente EpisodeRatingsGrid.jsx: grid visual de TODOS los episodios de la serie (puede ser cientos). Cada celda:

- Coloreada con gradiente de calor según rating TMDb: verde (>8.0), amarillo (6.0–7.9), rojo (<6.0), gris (sin datos).
- Tamaño pequeño (12×12 px) para visualizar patrones de calidad. ej. un episodio rojo en una fila de verdes es claramente el «peor» de la temporada.
- Click abre detalles del episodio.

Carga de ratings en paralelo: descargar ratings de 100+ episodios secuencia sería lentísimo. Solución: obtener todos los episodios de una temporada en paralelo mediante `Promise.all(getSeasonDetails(id, seasonNum) for each season)`. Resultado: latencia total 2–3 peticiones en lugar de 100.

4.4.4 4. Favoritos (`/favorites`)

Fuente: TMDb `GET /account/{id}/favorite/movies` y `/account/{id}/favorite/tv`.

Desafío de paginación: usuario puede tener 100+ favoritos (20 por página = 5+ páginas). Solución:

- Fetch página 1 para conocer `total_pages`.
- Fetch páginas restantes en lotes paralelos (máx. 5 simultáneas para respetar *rate limit*).
- Resultado: 5 páginas descargadas en tiempo de 2 peticiones paralelas en lugar de 5 secuenciales.

3 vistas alternativas:

- **Grid:** tarjetas 6 columnas en desktop, 4 en tablet, 2 en mobile. Poster + título + rating.
- **List:** filas horizontales con más información: sinopsis, año, géneros.
- **Compact:** lista densa una línea por ítem, optimizada para listas largas.

Selección de vista persiste en `localStorage['showverse:view:favorites']`.

Caché multinivel de puntuaciones:

1) **Nivel 1 (sessionStorage):** OMDb cache con TTL 24h. Si usuario está viendo favoritos y luego va a watchlist, los scores de OMDb que ya cargó persisten.
2) **Nivel 2 (deduplicación en vuelo):** Si 5 tarjetas solicitan `imdbId=tt123` simultáneamente, solo se realiza 1 fetch real a OMDb, todos esperan el resultado compartido.
3) **Nivel 3 (Lazy loading):** Solo cargar OMDb/Trakt scores al primer *hover* sobre tarjeta, no al montar página.

Función `runPool(items, limit, worker)`: ejecuta `worker(item)` para cada ítem manteniendo máximo `limit` operaciones concurrentes (típicamente 4), respetando *rate limits*.

Ordenación: fecha de adición (default), título alfabético, año, rating TMDb. **Filtro:** mostrar solo películas / solo series.

4.4.5 5. Pendientes/Watchlist (`/watchlist`)

Arquitectura idéntica a Favoritos, con diferencias mínimas:

- Endpoint: `/account/{id}/watchlist/movies` (vs. `/favorite/...`).
- Semántica: contenido que el usuario QUIERE ver (vs. YA ha visto).
- Ordenación adicional: los usuarios suelen reordenar watchlist por prioridad (qué ver primero).

Caché compartida entre Favoritos y Watchlist en módulo `scoreCache.js`.

4.4.6 6. Historial de visionado (`/history`)

Fuente principal: Trakt `/sync/history` (en lugar de TMDb).

Ventaja de Trakt: proporciona timestamp exacto de cada visionado (ISO 8601), hora, e incluso para series el episodio exacto.

Desafío: Trakt devuelve solo el ID de Trakt del contenido. Solución en API Route `/api/trakt/history`:

- Parsear visionados de Trakt.
- Para cada ítem: buscar su equivalente en TMDb usando `/search/tmdb/{traktId}` de Trakt.
- Hacer petición paralela a TMDb para obtener poster, backdrop, sinopsis.
- Enviar al cliente ya enriquecido.

UI: mismas 3 vistas que Favoritos. Mostrar fecha/hora de visionado próximo a cada ítem. Ordenación por defecto: más reciente primero.

4.4.7 7. En Progreso (`/in-progress`)

Lógica de cálculo: serie está «en progreso» si y solo si:

- Al menos 1 episodio está marcado como visto.
- No todos los episodios disponibles están marcados.

Implementación: parsear `/sync/watched/shows` de Trakt, comparar episodios vistos vs. totales para cada serie.

UI: mostrar para cada serie:

- Poster + título.
- Barra de progreso (ej. «30 / 62 episodios»).
- Botón «Continuar viendo» que linkea al siguiente episodio no visto.

4.4.8 8. Estadísticas del usuario (`/stats`)

Fuente: Trakt `/users/me/stats` + parseo de `/sync/history` para análisis profundos.

Visualizaciones:

- **Resumen global:** total películas vistas, series seguidas, episodios vistos, tiempo total invertido.
- **Gráfico de géneros:** BarChart (Recharts) mostrando distribución de consumo por género.
- **Heatmap de actividad:** tipo GitHub, mostrando actividad de visionado por día en última año (52 semanas).
- **Distribución de ratings:** histograma 1–10, mostrando cuántos títulos han sido puntuados en cada nivel.
- **Proporción películas/series:** gráfico de pie.

Performance: caché ISR de 1 hora (`revalidate = 3600`) ya que generar gráficos es costoso.

4.4.9 9. Calendario de estrenos (`/calendar`)

Fuentes combinadas:

- TMDb `/movie/upcoming` + `/tv/on_the_air`: estrenos generales.
- Trakt `/calendars/my/shows` (autenticado): próximos episodios de series que usuario sigue.

3 vistas de calendario:

1) **Día:** qué estrena HOY, actualización real-time.
2) **Semana:** próximos 7 días, filas por día.
3) **Mes:** calendario clásico tipo Google Calendar, celdas mostrando ítems de ese día.

Filtrado: por tipo (películas/series), período (hoy/semana/mes/próximos 30 días). **Ordenación:** por fecha (default) o popularidad.

4.4.10 10. Biblioteca Personal (`/biblioteca`)

Propósito: hub unificado personal, combinando:

- Resumen: última película/serie visto, series activas, favoritos recientes.
- Sección «Continuar viendo»: series en progreso con barra de progreso + botón al siguiente episodio.
- Sección «Historial reciente»: últimos 20 ítems vistos.
- Sección «Favoritos destacados»: últimos 12 favoritos.

Es esencialmente agregación de 3 módulos (Historial + En Progreso + Favoritos) en una página.

4.4.11 11. Descubrir contenido (`/discover`)

Motor de búsqueda avanzada:

- **Búsqueda libre:** debounce 400 ms sobre `/search/multi` de TMDb.
- **Filtros:** género (multiselect), año (rango), rating mínimo (rango), tipo (película/serie/persona).
- **Ordenación:** popularidad, rating, fecha lanzamiento, alfabético.

Implementación:

```
useEffect(() => {
  const timer = setTimeout(async () => {
    const results = await tmdbFetch('/search/multi', {
      query, page, with_genres, ...filters
    }, abortController.signal)
    setResults(results)
  }, 400)
  return () => clearTimeout(timer)
}, [query, filters])
```

AbortController: si usuario cambia página mientras hay búsqueda pendiente, cancela el fetch anterior para no procesar datos obsoletos.

Paginación infinita: `IntersectionObserver` detecta cuando usuario llega al final, carga página siguiente.

3 vistas: Grid, List, Compact (igual que Favoritos).

4.4.12 12. Gestión de listas (`/lists`)

4 tipos de listas:

1) **Listas propias del usuario en TMDb:** creadas/seguidas por usuario, editables.
2) **Listas curadas públicas de TMDb:** «Top 250 películas de todos los tiempos», «Mejor terror 2024», etc.
3) **Colecciones de saga:** franquicias como MCU, Star Wars (endpoint `/collection/{id}` de TMDb).
4) **Listas públicas de Trakt:** trending, recomendadas por comunidad.

Hub principal (`/lists`): agrupa todas bajo secciones. Cada lista tiene página individual (`/lists/[listId]`) con paginación y 3 vistas.

4.4.13 13. Películas y Series (`/movies`, `/series`)

Propósito: vistas temáticas filtradas por tipo único de contenido. Similar a dashboard pero sin secciones personalizadas, más géneros cubiertos (12+ vs. 3 en dashboard).

Organización: tendencias, top rated, populares, 8+ géneros (acción, drama, comedia, terror, etc.).

4.4.14 14. Búsqueda de personas (`/details/person/[id]`)

Datos:

- Biografía, fecha nacimiento, lugar nacimiento, foto.
- Filmografía completa como actor.
- Trabajos como director, guionista, productor.

UI: header con foto + datos, luego pestañas para Actor/Director/Guionista, cada una listando trabajos ordenados por popularidad, filtrable.

4.4.15 15. Barra de navegación (Navbar)

Ubicación constante: presente en todas las páginas. Gestiona:

- **Links de navegación:** a todas las secciones principales.

- **Busqueda rápida:** input con debounce que redirige a `/discover?q=...`.
- **Autenticación:** botones de login/logout para TMDb y Trakt.
- **Contadores:** número de favoritos, watchlist.
- **Avatar:** foto del usuario autenticado.
- **Efecto sticky:** transparente en top, glassmorphism después de 100 px scroll.

Responsive:

- **Desktop:** barra horizontal superior con 3 zonas (logo, búsqueda centro, links derecha).
- **Mobile:** barra superior simplificada + tab bar inferior con 4 secciones principales + drawer lateral para menú completo.

4.4.16 Patrones de optimización transversales

Los 15 módulos utilizan sistemáticamente los siguientes patrones:

1. Actualización optimista: al hacer clic en favorito/watchlist/rating, el estado visual se actualiza INSTANTÁNEAMENTE. Request al servidor en background. Si falla, revert automático. Elimina latencia perceptible (300–600 ms).

2. Lazy loading + caché: OMDb scores, Trakt ratings solo cargan al primer *hover*. Cacheados en sessionStorage/localStorage. Segunda vez, acceso instantáneo sin red.

3. Paginación paralela: descargar N páginas en paralelo en lugar de secuencial, con control de concurrencia máxima (4–5 simuláneas) para respetar *rate limits*. Resultado: 3–5x más rápido.

4. Deduplicación en vuelo: si múltiples usuarios/componentes piden mismo dato simultáneamente, una sola petición real, todos esperan su resultado.

5. Debounce en búsqueda: esperar 400 ms sin cambios antes de fetch. Limita peticiones a TMDb.

6. AbortController para cancelación: si usuario navega a otra página, cancela fetches en vuelo.

7. Pool de concurrencia: limitar máximo de requests simultáneos (4) para no saturar cliente ni servidor.

8. ISR con revalidación: datos «estables» (trending, top rated) cacheados en servidor 10–60 min, regenerados automáticamente. Balance entre frescura y velocidad.

9. 3 vistas alternativas: Grid, List, Compact. Selección persiste en localStorage. Flexibilidad UX.

4.5 Diseño de la interfaz

4.5.1 Sistema de diseño global

El sistema de diseño de *The Show Verse* parte de un principio rector: el contenido visual proveniente de TMDb (los *backdrops* y los *posters* de cada título) es el protagonista de la interfaz. La paleta de colores, los efectos de superficie y las estrategias de iluminación se han diseñado para potenciar, no competir con, esas imágenes.

Paleta y color de base. El color de fondo global es el negro puro (`#000000`), definido en el `layout.jsx` raíz. Sobre este fondo se despliegan los *backdrops* a pantalla completa con un gradiente perimetral que los integra suavemente en el negro sin corte brusco. Este enfoque traslada la estética de las propias plataformas de *streaming* (modo teatro) a la interfaz de seguimiento.

Superficies de cristal (*glassmorphism*). Las tarjetas, paneles laterales, la Navbar con *scroll* y los *dropdowns* de búsqueda utilizan el patrón definido en la clase utilitaria `.glass` del fichero `globals.css`:

```
.glass {
  background: rgba(255, 255, 255, 0.05);
  backdrop-filter: blur(10px);
  -webkit-backdrop-filter: blur(10px);
  border: 1px solid rgba(255, 255, 255, 0.1);
}
```

La opacidad del fondo (`0.05`) es deliberadamente baja para mantener la legibilidad del texto proyectando el color del *backdrop* subyacente en la superficie del elemento. El desenfoque de contexto (`backdrop-filter: blur`) es processing GPU-acelerado y no genera coste de repintado del DOM.

Tipografía. El proyecto usa la familia Geist Sans y Geist Mono, tipografías de Vercel distribuidas como variable CSS mediante el módulo `next/font`. Se cargan como fuentes de sistema sin peticiones externas, evitando el FOUT (*Flash of Unstyled Text*).

Accesibilidad y movimiento reducido. El fichero `globals.css` incluye una media query `prefers-reduced-motion` que desactiva todas las transiciones y animaciones de CSS para usuarios que han solicitado minimizar el movimiento en su sistema operativo, sin modificar ningún componente individual.

4.5.2 Componentes de Next.js utilizados

`<Image>` de Next.js. Todos los pósters, *backdrops* y logos de la aplicación se renderizan mediante el componente `<Image>` de Next.js en lugar del elemento `<img>` nativo. Este componente aplica automáticamente:

- Conversión del formato original JPEG a WebP para los navegadores que lo soportan, reduciendo el peso de las imágenes entre un 25 y un 35 % sin pérdida visual apreciable.
- *Lazy loading* nativo: las imágenes fuera del *viewport* no se solicitan hasta que el usuario se aproxima a ellas.
- Reserva de espacio (*placeholder*) mediante `width` y `height` explícitos, evitando el salto de layout (CLS) que se produciría si la imagen tardase en cargar.
- Optimización automática a través del CDN de Vercel, que sirve cada imagen en el tamaño exacto requerido por el dispositivo del usuario.

`<Link>` de Next.js. La navegación entre páginas utiliza el componente `<Link>` del App Router de Next.js, que implementa *prefetching* automático de las rutas visibles en el *viewport*. En producción (Vercel), cuando el usuario se aproxima a un enlace, Next.js precarga en segundo plano los datos del Server Component de la ruta destino, logrando transiciones de página percibidas como instantáneas.

`usePathname` y estado activo en la Navbar. El componente `Navbar.jsx` utiliza el *hook* `usePathname()` de `next/navigation` para detectar la ruta activa y aplicar el estilo visual correspondiente a cada enlace de navegación sin necesidad de estado global. La función `isActive(href)` comprueba si la ruta actual comienza con el *href* del enlace, marcando toda la sección como activa aunque el usuario esté en una subruta.

4.5.3 Componentes propios: Navbar

El componente `Navbar.jsx` (30 KB) implementa una navegación adaptativa con dos modos de presentación.

Modo escritorio (lg+). Una barra horizontal fija en la parte superior con tres zonas:

- **Izquierda**: logotipo de la aplicación y navegación principal (Inicio, Películas, Series, Descubrir, Biblioteca).
- **Centro** (posición absoluta): campo de búsqueda en tiempo real con *dropdown* de resultados.

- **Derecha**: botones de acceso rápido a Estadísticas, Listas, Calendario, En Progreso, Historial, Favoritas y Pendientes; avatar del usuario autenticado.

Cada botón de icono utiliza un sistema de estilos por «tono de color» (`iconLinkClass(href, tone)`): al hacer *hover*, el botón adquiere el color semántico de su función (rojo para Favoritas, azul para Pendientes, ámbar para Calendario, verde para En Progreso, etc.) junto con un halo de sombra coloreado:

```
hover:shadow-[0_0_18px_rgba(239,68,68,0.16)]
```

Cuando el estado está activo (ruta actual), el color se intensifica con un fondo semitransparente y una sombra más pronunciada.

Modo móvil. La barra superior se simplifica al logotipo centrado, un botón de menú (hamburgesa) y un botón de búsqueda. En la parte inferior de la pantalla aparece una barra de navegación fija (*tab bar*) con las cuatro secciones de mayor uso. Esta arquitectura de doble barra es el patrón estándar de las aplicaciones de contenido multimedia en iOS y Android.

El **drawer lateral** del menú móvil se implementa con Framer Motion:

```
<motion.aside
  initial={{ x: -320 }}
  animate={{ x: 0 }}
  exit={{ x: -320 }}
  transition={{ type: "tween", duration: 0.22 }}
>
```

El fondo oscuro detrás del drawer (`bg-black/70 backdrop-blur-sm`) también se anima con `AnimatePresence` para que tanto la entrada como la salida del menú sean fluidas.

La **búsqueda en móvil** abre un *overlay* a pantalla completa (`bg-black/95 backdrop-blur-2xl`) que entra con una animación de escala y opacidad (`scale: 0.95→1, opacity: 0→1`), dando la sensación de que emerge desde el botón de búsqueda.

4.5.4 Componente propio: LiquidButton

`LiquidButton.jsx` (18 KB) es el componente de botón de acción rápida utilizado en la página de detalle para las operaciones de favoritos, pendientes, historial y valoración personal. Es un Client Component completamente autónomo que implementa un sistema de efectos visuales interactivos mediante la API `Canvas 2D` del navegador.

La arquitectura del componente se basa en tres capas superpuestas dentro de un `<button>`:

1) **Canvas de efectos**: un elemento `<canvas>` que ocupa toda la superficie del botón y gestiona, mediante `requestAnimationFrame`, las partículas flotantes, las ondulaciones de *ripple* y los efectos de explosión al hacer clic.

2) **Superposición de brillo**: un `<div>` con un gradiente lineal animado mediante CSS (`@keyframes liquidShine`) que recorre la superficie del botón de forma continua cuando está activo o en *hover*.

3) **Borde pulsante**: un segundo `<div>` circular con borde coloreado animado mediante `@keyframes liquidPulse` (escala 1 → 1.15 → 1, con periodo de 2 s).

El sistema de efectos de *canvas* implementa:

- **Partículas flotantes**: 12 partículas con movimiento senoidal suave que se generan al entrar en *hover*. Cada partícula tiene posición, velocidad, tamaño, opacidad y fase trigonométrica independientes.
- **Ripple (ondulación)**: al hacer *hover* o clic, se crean ondas circulares que se expanden desde el punto de contacto y se desvanecen al alcanzar su radio máximo.
- **Trail effect**: al mover el ratón sobre el botón, se generan puntos de rastro que se desvanecen en 0,4 s, creando un efecto de «estela líquida».
- **Explosión**: al hacer clic, se generan 12 partículas que se dispersan en ángulos uniformes desde el punto de impacto con gravedad simulada (`vy += 0.12`) y decaimiento de opacidad.

El componente soporta un sistema de **propagación entre botones del mismo grupo** (`groupId`): al interactuar con un botón, este emite un `CustomEvent` al documento (`liquid-spread`, `liquid-click`, `liquid-proximity`) que es capturado por todos los botones del mismo grupo. Los botones próximos a la fuente del evento reaccionan con ondulaciones de intensidad proporcional a la distancia, creando la sensación de que el líquido se propaga físicamente entre botones adyacentes.

El color activo del botón es configurable mediante la prop `activeColor` (blue, red, yellow, purple, green, teal, orange), lo que permite usar el mismo componente para acciones semánticamente distintas manteniendo coherencia visual.

4.5.5 Componente propio: StarRating

`StarRating.jsx` gestiona la valoración personal del usuario en Trakt (escala 1–10). Utiliza `LiquidButton` como *trigger* con `activeColor=ƨellow"`: si el usuario ya tiene una

valoración, el botón muestra el valor numérico con el efecto de partículas activo; si no, muestra el icono de estrella.

Al hacer clic, el componente abre un modal renderizado mediante `createPortal()` directamente en `document.body`, evitando problemas de apilamiento CSS (`z-index`) con los elementos de la página de detalle. El modal contiene:

- Un número grande que muestra el valor actual centrado sobre una estrella decorativa semitransparente.
- Un *slider* HTML nativo (`<input type=range»`) con paso de 0,5 y recorrido de 1 a 10, cuyo *thumb* nativo se oculta con `opacity: 0` y se sustituye por un *thumb* personalizado posicionado con CSS (`left: calc({percentage}% - 14px)`).
- Botones de ajuste fino $-0{,}5$ / $+0{,}5$ a ambos lados del *slider*.
- Botón de confirmación y, si ya existe una valoración, botón de borrado con icono de papelera.

4.5.6 Componente propio: FavoriteWatchlistButtons

`FavoriteWatchlistButtons.jsx` encapsula los botones de Favorito y Pendiente de la página de detalle. Gestiona el estado de forma optimista: al pulsar el botón, el estado visual cambia inmediatamente (sin esperar la respuesta del servidor), y si la petición a TMDb falla, el estado se revierte y se muestra un mensaje de error. Este patrón elimina la latencia perceptible (normalmente de 300–600 ms) entre la acción del usuario y la respuesta de la interfaz.

Los iconos de los botones cambian según el estado (Heart / HeartOff, BookmarkPlus / BookmarkMinus), siguiendo el principio de retroalimentación directa. Cuando hay una operación en curso, los iconos se sustituyen por un spinner (`Loader2 animate-spin`) de Lucide React.

4.5.7 Sistema de animaciones con Framer Motion

Framer Motion (Framer, 2024) gestiona las animaciones de mayor complejidad del proyecto a través de cuatro patrones principales:

1. Animaciones de entrada escalonada (`staggerChildren`). En el dashboard y en las páginas de listas, las tarjetas no aparecen todas simultáneamente. La variante del contenedor define un `staggerChildren` de 0,05 s (reducido a 0,02 s para listas largas de más de 30 ítems), de forma que cada tarjeta entra con un ligero retraso respecto a la anterior:

```
const containerVariants = {
```

```
  hidden: { opacity: 0 },
  visible: {
    opacity: 1,
    transition: { staggerChildren: 0.05 }
  }
};
const itemVariants = {
  hidden: { opacity: 0, y: 20 },
  visible: { opacity: 1, y: 0 }
};
```

2. `AnimatePresence` para salidas animadas. El sistema de tres vistas (Grid, List, Compact) y las pestañas de la página de detalle utilizan `AnimatePresence` para animar tanto la entrada como la salida de los elementos del DOM. Cuando el usuario cambia de vista, el contenido saliente realiza una animación de salida (`exit`) antes de ser eliminado del DOM, evitando cortes bruscos:

```
<AnimatePresence mode="wait">
  <motion.div
    key={currentView}
    initial={{ opacity: 0, x: 20 }}
    animate={{ opacity: 1, x: 0 }}
    exit={{ opacity: 0, x: -20 }}
    transition={{ duration: 0.2 }}
  />
</AnimatePresence>
```

El modo `"wait"` garantiza que la animación de salida se complete antes de que empiece la de entrada.

3. Drawer y overlays móviles. El menú lateral, la búsqueda en *overlay* móvil y los paneles modales utilizan `motion.div` con valores iniciales fuera del *viewport*: `x: -320` para el drawer (deslizamiento desde la izquierda), `scale: 0.95, opacity: 0` para los modales (apertura con escala). Las transiciones de tipo `tween` con `ease: .easeOut"` producen una deceleración natural al final del movimiento.

4. Carrusel del hero. El carrusel automático de la página principal anima las transiciones entre slides con una combinación de `opacity` (entrada: 0→1, salida: 1→0) y un ligero desplazamiento horizontal (`x: +20→0`). La pausa automática al hacer *hover* se gestiona mediante un `useEffect` que detecta el evento `mouseenter` del contenedor del hero.

Performance de las animaciones. Todas las animaciones de Framer Motion utilizan exclusivamente las propiedades CSS `transform` (para `x`, `y`, `scale`, `rotate`) y `opacity`, que el navegador puede procesar enteramente en la GPU sin necesidad de recalcular el *layout* ni repintar capas del DOM. Esto garantiza que las animaciones se ejecuten a 60 fps incluso en dispositivos de gama media.

4.5.8 Sistema de estilos global (`globals.css`)

El fichero `globals.css` define un conjunto de utilidades y animaciones CSS que complementan a Tailwind CSS:

- **`.no-scrollbar`**: oculta la barra de desplazamiento en los carruseles horizontales y los paneles de pestañas, sin deshabilitar el *scroll* táctil.
- **`.sv-scroll`**: barra de desplazamiento personalizada para los paneles con mucho contenido (lista de episodios, resultados de búsqueda): fina, semitransparente, con bordes redondeados y espacio interior.
- **`.animate-shimmer`**: efecto de carga tipo *skeleton* con un gradiente que avanza horizontalmente (`background-position: -200% → 200%`) sobre los *placeholders* de las tarjetas mientras los datos se cargan.
- **`.hover-lift`**: eleva el elemento 4 px y añade una sombra al hacer *hover*, indicando que es interactivo.
- **`.glass / .glass-dark`**: superficies de *glassmorphism* en variante clara y oscura reutilizables en cualquier componente.

4.5.9 Sistema de tres vistas (Grid, List, Compact)

El sistema de tres vistas es un componente transversal utilizado en Favoritos, Pendientes, Historial y En Progreso. La selección de vista se persiste en `localStorage` bajo una clave específica por módulo (`showverse:view:favorites`, `showverse:view:history`, etc.), de forma que el usuario puede tener configuraciones distintas en cada sección.

La transición entre vistas utiliza `AnimatePresence` de Framer Motion con `mode="wait"` y un efecto de *cross-fade* suave: el contenido saliente desaparece con `opacity: 1→0` antes de que el nuevo contenido entre con `opacity: 0→1`. Esto evita los saltos bruscos de *layout* que se producirían si los dos estados coexistieran en el DOM simultáneamente.

4.6 Rendimiento en producción

Las métricas obtenidas en producción (Vercel) mediante Lighthouse validan las decisiones de arquitectura tomadas:

Métrica	**Valor obtenido**	**Objetivo**
Performance (Lighthouse)	92/100	>90
SEO (Lighthouse)	100/100	100
Best Practices	95/100	>90
LCP	1,8 s	<2,5 s
CLS	0,05	<0,1
FID	45 ms	<100 ms

Tabla 4.5: Métricas Lighthouse en producción

El LCP de 1,8 s se consigue principalmente gracias al SSR: el HTML con el *hero* ya renderizado llega al navegador antes de que se descargue todo el JavaScript. El CLS de 0,05 se logra mediante el uso sistemático de `width` y `height` explícitos en el componente `<Image>` de Next.js (Google Developers, 2024b).

Capítulo 5
Conclusiones

El trabajo ha demostrado que los objetivos planteados son alcanzables con las tecnologías actuales de desarrollo web y sin necesidad de infraestructura de servidor propia. Los resultados obtenidos permiten extraer las siguientes conclusiones.

En primer lugar, la arquitectura Next.js App Router con React Server Components ha demostrado ser particularmente adecuada para aplicaciones de este tipo, que combinan una gran cantidad de datos externos con la necesidad de interactividad en cliente. La separación entre la obtención de datos (servidor) y la presentación interactiva (cliente) simplifica el flujo de datos y reduce las fuentes de error. Los Server Components permiten realizar *fetching* paralelo de múltiples APIs en el servidor sin coste para el cliente, lo que resulta en tiempos de carga significativamente inferiores a los de una SPA equivalente.

En segundo lugar, la integración de OAuth 2.0 mediante API Routes de Next.js ha probado ser una estrategia robusta y segura sin necesidad de librerías de autenticación genéricas. El patrón de almacenar tokens en cookies `httpOnly` y delegar toda la lógica de autenticación al servidor cumple con las recomendaciones de seguridad del RFC 6749 (Hardt, 2012) y las guías de OWASP (OWASP, 2024) para aplicaciones OAuth.

En tercer lugar, la estrategia de caché diferenciada por tipo de dato ha demostrado ser eficaz en todas sus capas: ISR en servidor para el catálogo, `no-store` para los datos de usuario protegidos por autenticación, y una caché en cliente de tres niveles para las puntuaciones externas de IMDb, Trakt y OMDb en las vistas de listas, reduciendo drásticamente el número de llamadas a APIs externas.

Finalmente, desde el punto de vista del diseño, el uso de Framer Motion para las animaciones y el sistema de superficies semi-transparentes con desenfoque de fondo ha producido una interfaz visualmente diferenciada respecto a las plataformas de *tracking* existentes. Las métricas de rendimiento obtenidas son completamente compatibles con un diseño visualmente rico, siempre que las animaciones se implementen sobre la GPU (propiedades `transform` y `opacity`) y no fuercen la recarga del DOM.

5.1 Discusión

El trabajo presenta algunas limitaciones que deben reconocerse.

Ausencia de tests automatizados. El proyecto no incluyó una suite de tests unitarios ni de integración durante el desarrollo. La validación se realizó exclusivamente mediante pruebas manuales. Los componentes con lógica de negocio compleja (cálculo de progreso de episodios, normalización de fechas para Trakt, estrategia de selección de *backdrops*) serían candidatos prioritarios para la cobertura de tests unitarios.

Dependencia de APIs de terceros. La plataforma depende de la continuidad y estabilidad de cinco APIs externas (TMDb, Trakt.tv, OMDb, JustWatch y Plex), ninguna de las cuales está bajo el control del proyecto. Esta es una limitación estructural de la arquitectura elegida, aceptada conscientemente por las ventajas que ofrece frente a la alternativa de construir una base de datos propia.

Accesibilidad parcial. La puntuación de accesibilidad en Lighthouse (88/100) indica que, aunque se han aplicado prácticas básicas, no se ha alcanzado el nivel WCAG 2.1 AA completo (W3C, 2023). En particular, la navegación exclusiva por teclado en los modales de episodios requiere trabajo adicional.

5.2 Líneas de investigación futuras

A partir del trabajo realizado, se identifican las siguientes líneas de desarrollo futuro:

Implementación de Progressive Web App (PWA). La conversión de la aplicación en una PWA, mediante la adición de un *Service Worker* y un *manifest* web, permitiría la instalación en dispositivos móviles y habilitaría un modo de funcionamiento *offline* básico.

Sistema de recomendaciones personalizado. El historial de visionado acumulado por el usuario constituye un conjunto de datos con potencial para la implementación de algoritmos de recomendación colaborativa o basada en contenido, posiblemente mediante TensorFlow.js en el cliente o una función *serverless*.

Expansión de las funcionalidades sociales. La comparación del historial entre usuarios, las recomendaciones de amigos y la visualización del perfil público de otros usuarios de Trakt son funcionalidades que la API de Trakt soporta pero no han sido implementadas en la versión actual.

Backend propio y base de datos local. A largo plazo, la eliminación de la dependencia total de Trakt.tv como fuente de verdad requeriría la implementación de un *backend* propio con base de datos (PostgreSQL o MySQL) que almacenara el historial y las preferencias localmente.

Bibliografía

Babich, N. (2021). *Glassmorphism in User Interfaces*. UX Planet. https://uxplanet.org/glassmorphism-in-user-interfaces-1f39bb1308c9

CNMC. (2024). *Informe sobre el consumo de vídeo en España*. https://www.cnmc.es/

Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures. *Doctoral Dissertation, University of California*. https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm

Framer. (2024). *Framer Motion Documentation*. https://www.framer.com/motion/

Google Developers. (2024a). *Core Web Vitals report*. https://support.google.com/webmasters/answer/9205520

Google Developers. (2024b). *Web Vitals — Essential metrics for a healthy site*. https://web.dev/vitals/

Hardt, D. (2012). *RFC 6749: The OAuth 2.0 Authorization Framework*. IETF. https://www.rfc-editor.org/rfc/rfc6749

Letterboxd. (2024). *About Letterboxd*. https://letterboxd.com/about/

Meta Open Source. (2024). *React Documentation — React Server Components*. https://react.dev/reference/rsc/server-components

OWASP. (2024). *OWASP Top 10: Broken Authentication*. https://owasp.org/www-project-top-ten/

Schwartz, B. (2004). *The Paradox of Choice: Why More Is Less*. Ecco/HarperCollins.

Statista. (2024). *Number of Netflix subscribers worldwide from 1st quarter 2013 to 4th quarter 2023*. https://www.statista.com/statistics/250934/quarterly-number-of-netflix-streaming-subscribers-worldwide/

The Movie Database. (2024). *TMDb API Documentation v3*. https://developer.themoviedb.org/docs/getting-started

Trakt. (2024). *Trakt API Documentation v2*. https://trakt.docs.apiary.io/

Vercel. (2024). *Next.js Documentation — App Router*. https://nextjs.org/docs/app

W3C. (2023). *Web Content Accessibility Guidelines (WCAG) 2.1*. https://www.w3.org/TR/WCAG21/

Anexo

Variables de entorno del proyecto

La siguiente tabla recoge todas las variables de entorno necesarias para el despliegue de *The Show Verse*.

Variable	**Descripción**	**Requerida**
`NEXT_PUBLIC_TMDB_API_KEY`	API Key de TMDb v3	Sí
`TMDB_SESSION_ID`	Session ID para operaciones de cuenta TMDb	Cuentas
`NEXT_PUBLIC_TRAKT_CLIENT_ID`	Client ID de la aplicación Trakt	Sí
`TRAKT_CLIENT_SECRET`	Client Secret de Trakt (solo servidor)	Sí
`TRAKT_REDIRECT_URI`	URI de callback OAuth registrado en Trakt	Sí
`OMDB_API_KEY`	API Key de OMDb	Para IMDb/RT
`PLEX_URL`	URL del servidor Plex	Para Plex
`PLEX_URLS`	URLs adicionales Plex (separadas por coma)	Opcional
`PLEX_TOKEN`	Token de autenticación Plex (modo simple)	Modo simple
`PLEX_JWT_PRIVATE_KEY`	Clave privada RSA/Ed25519 (modo JWT)	Modo JWT
`PLEX_JWT_KID`	Key ID del JWK (modo JWT)	Modo JWT
`PLEX_CLIENT_IDENTIFIER`	Identificador único de la app en Plex	Modo JWT
`PLEX_MACHINE_IDENTIFIER`	Machine ID del servidor Plex (fallback)	Opcional

Tabla 5.1: Variables de entorno del proyecto

Resumen del sistema de caché multicapa

Nivel	**Almacenamiento**	**TTL**
1 – ISR servidor	Data Cache de Next.js	10 min / 1 h
2 – CDN Vercel	Edge Network	1 – 24 h
3 – Memoria proceso	`Map` en `globalThis`	24 h (éxito) / 60 s (fallo)
4 – `sessionStorage`	Navegador (sesión)	24 h
5 – `localStorage`	Navegador (persistente)	30 días
6 – `Map` en módulo	Memoria del cliente (tab)	10 min / 45 s

Tabla 5.2: Sistema de caché multicapa